ThaiSLM — file structure (v8)

Everything a person continuing the project needs: what every path holds, where the deployable model is, how to run the service, how to rebuild data and models. Last updated with the v8 release (`result_reporting/result_v8.md`).

1. Top level

path	what it is	in git?
`main.py`	**the only CLI entry point** — `python main.py` lists every command (infer · test · data · prep · extract · harvest · audit-youtube · train-encoder · train-seq · eval · export-onnx · bench-tts · diagnose · cloud · pretrained)	✓
`docker-compose.yml`	**web demo / service** — `docker compose up --build` → http://localhost:8080	✓
`.dockerignore`	keeps the Docker build context to code + the MediaPipe model (data and models never enter images)	✓
`requirements.txt`	Python 3.10 environment for training / development (conda env `hugging`)	✓
`README.md`	start-here guide	✓
`lab.ipynb`	post-training lab (v8): data audit, hand tracking, latent space / prior bias, tagger, segmentation, results on unseen people, speech, production (generated by `scripts/make_lab.py`)	✓
`.env`	secrets and local settings (OpenAI key, GCP project / bucket, AWS SSO profile) — **never committed**	✗
`.gitignore`, `.gitattributes`	what goes to git; model weights via Git LFS (`*.pt`, `*.npz`, `*.onnx`, `*.task` under `models/`)	✓

2. `models/` — the deployable model (load this to use ThaiSLM)

file	content
`mediapipe/holistic_landmarker.task`	the pose / hand / face extractor (MediaPipe Holistic, tasks 1.0.1) — identical file in Python and in the browser
`mediapipe/hand_landmarker.task`	hand fallback (MediaPipe HandLandmarker): fills hands Holistic misses near a raised pose wrist
`encoder.pt`	sign encoder: ensemble of three Thai-only landmark encoders (Conv1D + Transformer, 10.2 M parameters, 768-d), PyTorch
`tagger.pt`	frame tagger (BiGRU): non-sign / sign-begin / sign-inside
`span_head.npz`	one-pass spotting head (NumPy)
`bank.npz`, `concepts.json`	vocabulary bank (one embedding per labelled clip) and concept names
`vocab_daily.json`	**default** daily vocabulary (1,366 words with ≥ 2 signers + the conversation list)
`vocab_conversation.json`	327-word daily-conversation list (conversation mode)
`null_proto.npy`, `freq_prior.npy`	"not a sign" prototype, Thai word-frequency prior
`spotting.json`, `segment_calibration.json`	decoder and calibration settings per vocabulary
`grammar.json`, `roles.json`, `roles_language.json`	TSL word order + word roles (re-ranker) · roles for composing Thai sentences (daily words set by category)
`fingerspell.npz`, `letters_bank.npz`	fingerspelling recogniser (3 averaged NumPy MLPs, A–Z + ก–ฮ + 7 vowels) and the letter bank (encoder embeddings); settings in `spotting.json` "spell"
`face_template.npz`	mean face shape for the Standard Schema
`config.json`	provenance (encoder kind `lmenc`, runs, sizes, version)
`onnx/encoder.onnx` (43 MB, dynamic length), `onnx/tagger.onnx`, `onnx/parity.json`	production graphs for ONNX Runtime
`README.md`	how to load / extend the model

Use it: `SignPipeline(backend="onnx", device="cpu", vocab="daily").run("video.mp4")` (see `models/README.md`), or the web API.

3. `modules/` — the library

module	role	used at inference?
`mediapipe_pose.py`	MediaPipe Holistic extraction (video file) + HandLandmarker fallback, browser-landmark parsing, hand-tracking report → Standard Schema	✓
`schema.py`	Standard Schema (133-slot layout, 25 fps, face template, hand-gap filling), handedness canonicalisation, activity / sign span	✓
`lmenc.py`	**v8 encoder**: landmark features (122 / frame), Conv1D + Transformer model, ensemble, `EncoderAPI` (one interface for v8 and the Uni-Sign reference)	✓
`parts.py`	Standard Schema → the four part streams the Uni-Sign encoder reads (v7 / zero-shot comparisons)	comparisons
`runtime.py`	ONNX Runtime / PyTorch back-ends with one NumPy interface; vocabulary `Bank`; window tables; exact re-scoring	✓
`spanhead.py`	one-pass spotting head: NumPy inference + distillation training	✓
`segment.py`	kinematic features, windows, temporal resampling, tagger decoding, boundary metrics (NumPy)	✓
`decode.py`	semi-Markov decoder, word priors, metrics, calibration (logistic), transition / repeat post-processing	✓
`pipeline.py`	`SignPipeline`: everything end to end (+ result.json / subtitle.srt / timeline.png)	✓
`face.py`	face channel: blendshapes → question / negation / emotion (calibrated on neutral dictionary faces)	✓
`fingerspell.py`	fingerspelling: hand-shape window features, NumPy MLP, letter events	✓
`language.py`	evidence-guarded Thai sentence (OpenAI or rules)	✓
`grammar.py`	TSL word-order patterns (TSL51) + optional re-ranker	✓
`tts.py`	Thai speech (VachanaTTS default; MMS-TTS, edge, OpenAI optional; `.env` `TTS_PROVIDER`)	✓ (tts service)
`encoder.py`	training data sampler (P×K, hard negatives, cross-extractor positives, webcam + co-articulation augmentation), losses	training
`unisign.py`	Uni-Sign encoder (v7 / zero-shot reference only)	comparisons
`sequence.py`	tagger model, sentence synthesis, forced alignment, the local sequence stage	training
`lexicon.py`	gloss → lemma → concept (rules, synonym groups, data_test references)	data prep
`identity.py`	signer identity from faces (DINOv2) — data prep only	data prep
`utils.py`, `env.py`	paths (`THAISLM_MODELS`, `THAISLM_DATA`, `THAISLM_ARTIFACTS` overrides), logging, `.env` loader	✓

4. `scripts/` — pipeline steps (all reachable through `main.py`)

script	what it does
`data_prep.py`	raw data → `data_prep/` (video list, ingest of landmark-only sources, signers, lexicon, manifest + QC, training shards, SCHEMA.md)
`extract_mp.py`	MediaPipe on every source video (local CPU, resumable, two processes can share the work)
`audit_youtube.py`	v8 YouTube audit: `measure` (cuts, people, zoom, framing, hands, label agreement) · `sheets` (contact sheets for the eye check) · `decide` → `qc/quality_v8.csv`
`train_encoder.py`	v8 encoder training (resumable), ensemble, reference embeddings, isolated report, continuous-window probe
`harvest_words.py`	YouTube single-word lessons for daily words: discover → download (data_test duplicate check) → videos → signers (faces, data_test look-alike exclusion) → label (span + consistency)
`eval.py`	build `models/`, grammar, decoder tuning (`spot`, `tune_natural`), per-speed calibration (`calibrate`), final spotting report, data_test (`infer`), external videos (`external`), phrases, self-made sentence videos (`selftest`), vocab exports
`diagnose.py`	per-recording diagnosis: landmark / segmentation / not in vocabulary / confusable / sign differs → `result_reporting/diagnose/<name>/`
`bench_tts.py`	Thai TTS benchmark (CER by a Thai ASR, speed, pitch) → `artifacts/reports/tts_bench.json`
`replay_live.py`	replay saved live requests through the running service
`export_onnx.py`	`models/onnx/*.onnx` + parity check
`infer.py`	translate one video (CLI)
`make_lab.py`	writes / executes `lab.ipynb`
`harvest_letters.py`	alphabet lessons → letter holds (`agent_data/en_alpha`, `th_alpha`)
`train_fingerspell.py`	trains the fingerspelling recogniser + letter bank (CPU)
`tune_spelling.py`	letter settings on tune data, report on held-out data
`study_clips.py`	reference clips of each word / letter of a sentence (e.g. `data_test/jack_test/`)
`md_to_pdf.py`	Markdown report → PDF (headless Chrome)
`s3_sync.py`	S3 data lake pull (`python main.py data pull …`)
`pretrained.py`	rebuild `artifacts/pretrained/` (Uni-Sign base weights from Hugging Face; only for the zero-shot comparison)
`collect/`	the original raw-data harvester (TTRS / YouTube, v4) — kept to rebuild raw_data

5. `GCP/` — cloud GPU (Vertex AI)

path	content
`GCP/script/vertex.py`	`upload-code` · `upload-data` · `upload-models` · `launch` (same job in several regions, first to start wins, the rest are cancelled while pending) · `wait` · `status` · `cancel` · **`cleanup`** (empty the bucket, delete every job, check that no VM / disk / IP / endpoint bills)
`GCP/script/job_entry.py`	what runs inside the container (`train_encoder` ; `sequence` runs locally via `python main.py train-seq`)
`GCP/script/gcp_common.py`	`.env` → project / bucket / machine profiles, cost estimate
`GCP/script/follow_logs.py`	stream a running job's log lines
`GCP/script/requirements_job.txt`	extra pip packages inside the PyTorch container
`GCP/result/*.json`	one record per job: machine, region, queue / run minutes, estimated cost (v8 used no cloud job — empty)

6. `webapp/` — the web demo (micro-services)

path	service
`webapp/web/`	Next.js 16 front end (`components/SignStudio.tsx` camera / live mode / upload, `components/ResultCard.tsx` (+ hand-tracking footer), `lib/holistic.ts` MediaPipe Holistic + HandLandmarker fallback + GPU light filter, `lib/api.ts`)
`webapp/api/`	`sign-api` (FastAPI + ONNX Runtime) — `app.py` , `Dockerfile` , `requirements.txt`
`webapp/tts/`	`tts` (FastAPI + VachanaTTS, ONNX)
`webapp/gateway/nginx.conf`	gateway routes `/` → web, `/api/` → sign-api, `/tts/` → tts
`webapp/README.md`	how to run, API reference, scaling notes

7. `artifacts/` — training outputs that are not the deployable model

path	content
`pretrained/`	`mt5_config.json`; Uni-Sign base weights only when rebuilt (`python main.py pretrained`, not in git) for the zero-shot comparison
`runs/<run>/`	one folder per training run: `train_history.json`, `metrics_*.json`, `summary.json`, `job.log`; the local sequence run (`seq-v8/`): tagger, span head, alignment, `sequence_metrics.json` (per-clip arrays are not in git)
`eval/bank_eval.npz`	the train-only bank (honest evaluation: held-out signers are never in it)
`eval/grammar_tune.json`	TSL word-order patterns learned from tune templates only (decoder tuning)
`eval/youtube_roles_reference.json`	v7's train / val / test role of each YouTube clip (the "held out by both models" protocol)
`reports/*.json`	every evaluation. v8: `isolated_v8.json`, `continuous_windows_*.json`, `spotting.json` (tuning), `spotting_final_v8.json`, `natural_tuning.json`, `calibration_v8.json`, `infer_v8.json` (data_test), `external_v7.json` / `external_v8.json` + `external_test_list.tsv`, `phrases_v8.json`, `selftest_v8.json`, `tts_bench.json`; v7 baselines: `isolated.json`, `infer_final.json`, `phrases.json`, `selftest.json`; `nmm_validation.json` (face cues), `fingerspell.json`, `spelling.json`
`runs/`	only the v8 runs are kept: `enc-v8a/b/c` (members), `enc-v8` (ensemble), `seq-v8` (tagger, span head, alignment), `v7-on-v8` / `zeroshot-on-v8` (reference embeddings), `fingerspell`

8. Data (not in git — S3 `s3://dsi490-lake-signdata/`, local mirror `cloud_s3/`)

path	content
`cloud_s3/raw_data/`	raw videos / landmark releases (TTRS, th_sl, TSL-ONE-S, TSL51, YouTube)
`cloud_s3/data_prep/`	Standard Schema: `pose_mp/` + `pose_mp_raw/` (v7 MediaPipe), `pose/` (v6 RTMW + landmark-only sources), `manifest/`, `params/` (`video_items.csv`, signer clusters, `tsl51_alignment_v6.json` = transition source for the shards ...), `qc/`, `shards/` (training input), `SCHEMA.md`
`cloud_s3/agent_data/en_alpha/`, `th_alpha/`	fingerspelling lessons, landmarks, letter labels (`items.csv`), README per folder
`cloud_s3/agent_data/youtube_words/`	v7 harvest (v8 audit in `qc/`: `measure_v8.csv`, `eyecheck_v8.csv`, `quality_v8.csv`, `sheets_v8/`): `videos/` + `meta/` (yt metadata), `candidates.csv`, `phrases.csv`, `download_status.csv`, `videos.csv`, `signers.csv`, **`items.csv` (the annotations: word, concept, span, signer, role, status + reason)**, `qc/` (signer contact sheet, identity report)
`data_test/`	5 real-world test videos (never used for training or tuning)
`cache/`	scratch, not in git: MediaPipe of data_test (`data_test_mp/`), self-made test videos (`selftest/`), `prep_legacy/` (v5-era intermediates `prep ingest` still reads), `thsl_face_emb.npz`, `live_requests/` + `replay_src/` (your saved live utterances, landmarks only), `shards_mm/` (rebuilt automatically by `train-encoder`)

9. `vocab/` and `result_reporting/`

path	content
`vocab/daily_conversation.json`	the ~340-word daily-conversation target list (by category)
`vocab/daily_conversation_depth.csv`	signers per daily word
`vocab/missing_word_strategy.json`	how words with no Thai data are covered (Thai synonym / composition / face / fingerspelling — no foreign signs)
`vocab/vocab.csv`	every concept: instances, signers, datasets, role, in the conversation vocabulary
`result_reporting/result_vN.md/.pdf`	reports per version (v5, v6, v7, v8)
`result_reporting/architecture_vN`, `latent_space_vN`	architecture / representation reports
`result_reporting/diagnose/<name>/`	example `diagnose` output (report.md, tracking.png, result.json)
`result_reporting/inference/<vocab>/<video>/`	data_test outputs (result.json, subtitle.srt, timeline.png)
`result_reporting/file_structure.md/.pdf`	this document

10. Common tasks

```
# run the service
docker compose up --build                                    # http://localhost:8080
docker compose up --build --scale sign-api=3                 # more API replicas behind the gateway
THAISLM_MODELS_DIR=./other_models docker compose up          # serve another model folder

# use the model from Python / CLI
python main.py infer --video clip.mp4 --vocab daily --tts
python main.py diagnose <recording.json.gz> --intended "พ่อ ดื่ม น้ำ"     # why a recording was misread
python main.py test                                          # data_test end-to-end

# add a word / a signer without retraining: extract → embed → append to models/bank.npz (models/README.md)

# rebuild everything
python main.py data pull data_prep                           # or rebuild: prep pose_items → extract → prep
ingest/lexicon/manifest
python main.py audit-youtube measure && python main.py audit-youtube decide
python main.py prep shards
python main.py train-encoder --run enc-v8a   # + enc-v8b (--ema 0.998), enc-v8c (--context_p 0.5 --jitter 0.3); laptop
GPU, resumable
python main.py train-encoder --ensemble enc-v8a,enc-v8b,enc-v8c --run enc-v8
python main.py train-seq --encoder_run artifacts/runs/enc-v8
python main.py eval build --run enc-v8 --seq seq-v8 && python main.py eval grammar && python main.py eval spot --seq
seq-v8
python main.py eval tune_natural --seq seq-v8 && python main.py eval calibrate --seq seq-v8
python main.py export-onnx && python main.py train-letters --bank_only && python main.py test && python
scripts/make_lab.py --execute
python main.py cloud cleanup                                 # ALWAYS after a cloud GPU round (full list: result_v8
§11)
```

11. Workspace clean-up (v8)

v7 leftovers that nothing reads any more were moved out of the repository to `D:\SLM-labs\_thaislm_v8_cleanup\` (v7 runs `enc-v7-*` / `seq-v7`, the v7 model copy used for comparisons, v6

embeddings, v7 GCP job records, v7 report figures and their script, the Uni-Sign base weights, the memory-mapped shard cache, training logs, report drafts, TTS benchmark audio). Delete that folder (and `D:\SLM-labs\_thaislm_v7_cleanup\`) when you no longer need them; v7 itself stays recoverable from git (`86098cb`). The external test videos were deleted after the test (only the list, `artifacts/reports/external_test_list.tsv`, is kept). The v5 – v7 reports (`.md` + `.pdf`) are kept.